

Kubernetes -01

Tasks:

1. Setup Minikube in your local machine.
2. Setup k8s master and two worker nodes on ubuntu.
3. Run one nginx pod.
4. Mugup Master and slave components on k8s.

What is Kubernetes (K8s) ?

—>Kubernetes is also called as k8s.

—> Kubernetes is an open-source container orchestration platform used to deploy, manage, scale and monitor containerized applications automatically across multiple machines.

—> **Kubernetes** is a tool that **automatically manages containers** (like Docker containers).

Think simple:

Docker = runs one container

Kubernetes = controls many containers on many computers

So it is like a **manager of containers**.

Task-1 : Setup Minikube in Local Machine

Step 1: Install Docker (container runtime)

—> iam using macOS to do this task below are the macOS steps

Docker is required because Minikube runs Kubernetes inside a container.

```
brew install --cask docker
```

Start Docker Desktop from Applications → wait until it shows **Docker is running**

Verify:

```
docker --version
```

```
waseemakram@waseem ~ % docker --version
Docker version 28.5.1, build e180ab8
waseemakram@waseem ~ %
```

Step 2: Install kubectl (Kubernetes CLI)

kubectl is used to control Kubernetes cluster.

```
brew install kubectl
```

Verify:

```
kubectl version --client
```

```
waseemakram@waseem ~ % kubectl version --client

Client Version: v1.34.1
Kustomize Version: v5.7.1
```

Step 3: Install Minikube

Minikube creates a single-node Kubernetes cluster locally.

```
brew install minikube
```

Verify:

```
minikube version
```

```
waseemakram@waseem ~ % brew install minikube
=> Auto-updating Homebrew...
Adjust how often this is run with `HOMEBREW_AUTO_UPDATE_SECS` or disable with
`HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
=> Auto-updated Homebrew!
Updated 3 taps (hashicorp/tap, homebrew/core and homebrew/cask).
=> New Formulae
bagel: CLI to audit posture and evaluate compromise blast radius
bazel@8: Google's own build tool
difi: Pixel-perfect terminal diff viewer
go@1.25: Open source programming language to build simple/reliable/efficient software
happy-coder: CLI for operating AI coding agents from mobile devices
libunwind: C++ base UWP library derived from Portable UWP C++ library
```

```
waseemakram@waseem ~ % minikube version

minikube version: v1.38.0
commit: de81223c61ab1bd97dcfcfa6d9d5c59e5da4a0cf
waseemakram@waseem ~ %
```

Step 4: Start Minikube Cluster

This command creates the Kubernetes control-plane node locally.

```
minikube start
```

Expected Output:

Cluster started successfully and kubectl configured automatically.

```
[waseemakram@waseem ~ % minikube ip
🔧 Profile "minikube" not found. Run "minikube profile list" to view all profiles.
👉 To start a cluster, run: "minikube start"
[waseemakram@waseem ~ % minikube start
minikube v1.37.0 on Darwin 26.3 (arm64)
🌟 Automatically selected the docker driver
🎉 minikube 1.38.0 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.38.0
💡 To disable this notice, run: 'minikube config set WantUpdateNotification false'

🔧 Using Docker Desktop driver with root privileges
👉 Starting "minikube" primary control-plane node in "minikube" cluster
🔧 Pulling base image v0.0.48 ...
📦 Downloading Kubernetes v1.34.0 preload ...
> preloaded-images-k8s-v18-v1...: 332.38 MiB / 332.38 MiB 100.00% 12.21 M
> gcr.io/k8s-minikube/kicbase...: 450.06 MiB / 450.06 MiB 100.00% 7.70 Mi
🔥 Creating docker container (CPUs=2, Memory=3072MB) ...
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
[waseemakram@waseem ~ % minikube ip
192.168.49.2
[waseemakram@waseem ~ % minikube ps
```

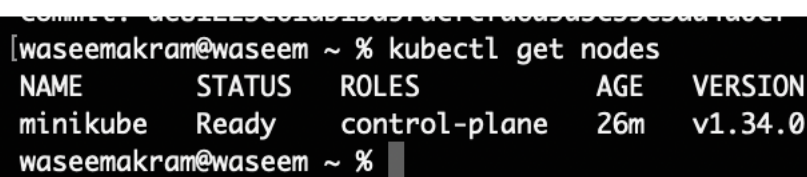
Step 5: Verify Cluster

Check node status:

```
kubectl get nodes
```

Output:

```
minikube    Ready    control-plane
```

A terminal window with a black background and white text. The prompt is 'waseemakram@waseem ~ %'. The command 'kubectl get nodes' has been executed, resulting in a table with 5 columns: NAME, STATUS, ROLES, AGE, and VERSION. The table contains one row for 'minikube' with status 'Ready', role 'control-plane', age '26m', and version 'v1.34.0'.

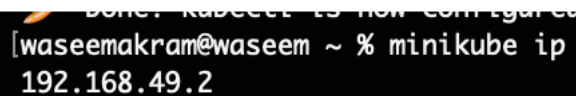
NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	26m	v1.34.0

Step 6: Get Cluster IP

```
minikube ip
```

Example:

```
192.168.49.2
```

A terminal window with a black background and white text. The prompt is 'waseemakram@waseem ~ %'. The command 'minikube ip' has been executed, resulting in the output '192.168.49.2'.

```
192.168.49.2
```

Step 7: Verify Kubernetes Components

Login into Minikube node:

```
minikube ssh
```

```
docker ps
```

Running containers observed:

- kube-apiserver
- kube-scheduler
- kube-controller-manager
- etcd
- kube-proxy
- coredns

This confirms Kubernetes master components are running.

```
[waseemakram@waseem ~ % minikube ssh
[docker@minikube:~$ docker images
REPOSITORY                                TAG          IMAGE ID       CREATED        SIZE
registry.k8s.io/kube-apiserver            v1.34.0      d291939e9940   5 months ago   83.7MB
registry.k8s.io/kube-scheduler            v1.34.0      a25f5ef9c34c   5 months ago   50.5MB
registry.k8s.io/kube-controller-manager    v1.34.0      996be7e86d9b   5 months ago   71.5MB
registry.k8s.io/kube-proxy                v1.34.0      6fc32d66c141   5 months ago   74.7MB
registry.k8s.io/etcd                      3.6.4-0      a1894772a478   6 months ago   205MB
registry.k8s.io/pause                    3.10.1       d7b100cd9a77   8 months ago   514kB
registry.k8s.io/coredns/coredns          v1.12.1      138784d87c9c   10 months ago  72.1MB
gcr.io/k8s-minikube/storage-provisioner    v5           ba04bb24b957   4 years ago    29MB
[docker@minikube:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                            CREATED        STATUS        PORTS
dae39c5fe9a9   ba04bb24b957                        "/storage-provisioner"                          About a minute ago   Up About a minute
d545671ba840   138784d87c9c                        "/coredns -conf /etc/"                         About a minute ago   Up About a minute
```

```
[docker@minikube:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                            CREATED        STATUS        PORTS        NAMES
dae39c5fe9a9   ba04bb24b957                        "/storage-provisioner"                          About a minute ago   Up About a minute   k8s_storage-provisioner_storage-provisioner_kube-system_187cb215-221b-42c9-be36-46826c7438d6_1
d545671ba840   138784d87c9c                        "/coredns -conf /etc/"                         About a minute ago   Up About a minute   k8s_coredns_coredns-66bc5c9577-kfn4k_kube-system_08b596b6-23ce-463c-9379-11eb5b3142ea_0
b3bf5e8e8987   registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_coredns-66bc5c9577-kfn4k_kube-system_08b596b6-23ce-463c-9379-11eb5b3142ea_0
9973752e1dc6   registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_kube-proxy_kube-proxy-thrqh_kube-system_eb52f400-609f-4a68-bc38-6c4f691ca5f3_0
6ef190d5f3be   registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_kube-proxy-thrqh_kube-system_eb52f400-609f-4a68-bc38-6c4f691ca5f3_0
14c64f06d778   registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_storage-provisioner_kube-system_187cb215-221b-42c9-be36-46826c7438d6_0
e168bbbd138    a1894772a478                        "etcd --advertise-cl..."                      About a minute ago   Up About a minute   k8s_etcd_etcd-minikube_kube-system_e3a36fac0ae701bc11fad0a6716ec2c_0
5a6c97758dc8    a25f5ef9c34c                        "kube-scheduler --au..."                      About a minute ago   Up About a minute   k8s_kube-scheduler_kube-scheduler-minikube_kube-system_dc6cf0a7bcb54d1f95cecc4d7b6b7d67_0
2ec97e8a7f35    996be7e86d9b                        "kube-controller-man..."                      About a minute ago   Up About a minute   k8s_kube-controller-manager_kube-controller-manager-minikube_kube-system_3b51c8241e224d47681cce32ea99b407_0
7c5fec9dd660    d291939e9940                        "kube-apiserver --ad..."                      About a minute ago   Up About a minute   k8s_kube-apiserver_kube-apiserver-minikube_kube-system_8312b4cdc4b705c0e12f63794469cfad_0
a300ba32ac4c    registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_kube-controller-manager-minikube_kube-system_3b51c8241e224d47681cce32ea99b407_0
85bd2068c0a6    registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_kube-scheduler-minikube_kube-system_dc6cf0a7bcb54d1f95cecc4d7b6b7d67_0
f9af28ad324e    registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_kube-apiserver-minikube_kube-system_8312b4cdc4b705c0e12f63794469cfad_0
991e6e3e2bb2    registry.k8s.io/pause:3.10.1        "/pause"                                          About a minute ago   Up About a minute   k8s_POD_etcd-minikube_kube-system_e3a36fac0ae701bc11fad0a6716ec2c_0
[docker@minikube:~$ minikube version
-bash: minikube: command not found
[docker@minikube:~$ exit
logout
ssh: Process exited with status 127
```

Result

Minikube single-node Kubernetes cluster successfully installed and verified on local machine.

Task:2

2.Setup k8s master and two worker nodes on ubuntu.

(Step1 to step 7) in all nodes : master,worker1,worker2 same steps

Step:1

Setup Kubernetes 1 Master + 2 Worker nodes on Ubuntu

Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents **Quick Start**

Amazon Linux macOS **Ubuntu** Windows Red Hat SUSE Linux Debian

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-06e3c045d79fd65d9 (64-bit (x86)) / ami-01da1dbf9ea3a6ee6 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description

Ubuntu Server 24.04 LTS (HVM),EB5 General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

Architecture **AMI ID** **Publish Date** **Username**

64-bit (x86) ami-06e3c045d79fd65d9 2025-12-12 ubuntu **Verified provider**

Summary

Number of instances Info
3

When launching more than 1 instance, [consider EC2 Auto Scaling](#)

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...[read more](#)
ami-06e3c045d79fd65d9

Virtual server type (instance type)
m7i-flex.large

Firewall (security group)
default

Storage (volumes)
1 volume(s) - 20 GiB

Cancel **Launch instance** [Preview code](#)

Instances (1/3) Info

Last updated 1 minute ago [Refresh](#) [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Find Instance by attribute or tag (case-sensitive)

running [Clear filters](#)

All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4
☐	k8s-master	i-0c3faabb470a01398	Running	m7i-flex.large	Initializing	View alarms	us-east-2a	ec2-18-116-37-147.us-...	18.116.37
☐	k8s-worker1	i-0b424c636f70f9196	Running	m7i-flex.large	Initializing	View alarms	us-east-2a	ec2-3-12-152-185.us-e...	3.12.152.1
☒	k8s-worker2	i-0c8a37205a734d495	Running	m7i-flex.large	Initializing	View alarms	us-east-2a	ec2-18-119-136-35.us-...	18.119.13

Step:2

2) Change the hostname

vi /etc/hostname

hostname master

—> sudo hostnamectl set-hostname master

similarly do it workers hostnames

```
[ubuntu@ip-172-31-14-17:~]$ sudo vi /etc/hostname
[ubuntu@ip-172-31-14-17:~]$ sudo cat /etc/hostname
master
ubuntu@ip-172-31-14-17:~$
```

```
ubuntu@ip-172-31-14-17:~$ sudo hostnamectl set-hostname master
ubuntu@ip-172-31-14-17:~$ hostname
master
ubuntu@ip-172-31-14-17:~$
```

After changing hostname refresh or login to your server again

—> it will show ubuntu@master

```
Last login: Mon Feb 16 10:02:43 2026 from 14.192.14.51
[ubuntu@master:~]$ hostname
master
ubuntu@master:~$
```

—> similarly do in other workers

-> **worker 1:(server)**

sudo hostnamectl set-hostname worker1

```
ubuntu@ip-172-31-9-88:~$ sudo hostnamectl set-hostname worker1
ubuntu@ip-172-31-9-88:~$ hostname
worker1
ubuntu@ip-172-31-9-88:~$ █
```

```
Last login: Mon Feb 16 10:18:58 2026 from 3.16.146.4
ubuntu@worker1:~$ hostname
worker1
ubuntu@worker1:~$ █
```

Worker-2 (server)

To check for new updates run: sudo apt update

```
Last login: Mon Feb 16 10:22:00 2026 from 3.16.146.4
ubuntu@worker2:~$ hostname
worker2
ubuntu@worker2:~$ █
```


STEP 3 — Disable swap

Do these on **ALL 3 nodes** (master + worker1 + worker2)

```
sudo swapoff -a
sudo sed -i ' / swap / s/^/#/' /etc/fstab
Verify:
```

```
free -h
```

Master:

```
master
[ubuntu@master:~$ sudo swapoff -a
[ubuntu@master:~$ sudo sed -i ' / swap / s/^/#/' /etc/fstab
ubuntu@master:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	483Mi	7.0Gi	2.9Mi	422Mi	7.1Gi
Swap:	0B	0B	0B			

```
ubuntu@master:~$ █
```

Worker2

```
ubuntu@worker2:~$ sudo swapoff -a
ubuntu@worker2:~$ sudo sed -i ' / swap / s/^/#/' /etc/fstab
ubuntu@worker2:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	466Mi	7.0Gi	2.9Mi	408Mi	7.1Gi
Swap:	0B	0B	0B			

```
ubuntu@worker2:~$ █
```

Worker1

```
ubuntu@worker1:~$ sudo swapoff -a
ubuntu@worker1:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	473Mi	7.0Gi	2.9Mi	414Mi	7.1Gi
Swap:	0B	0B	0B			

```
ubuntu@worker1:~$ sudo sed -i ' / swap / s/^/#/' /etc/fstab
ubuntu@worker1:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	472Mi	7.0Gi	2.9Mi	414Mi	7.1Gi
Swap:	0B	0B	0B			

```
ubuntu@worker1:~$ █
```

Step 4: Configure Host Resolution

Do these on **ALL 3 nodes** (master + worker1 + worker2)

Each node must communicate using hostnames.

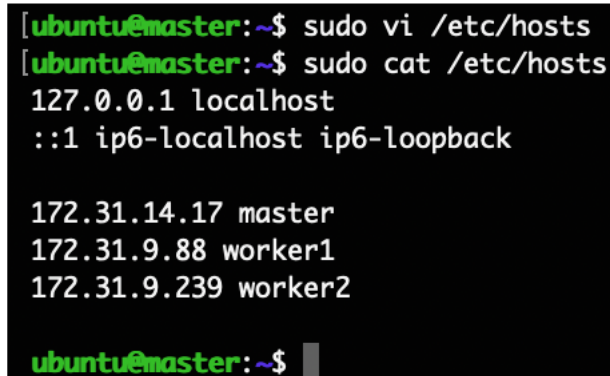
Edited hosts file on all machines:

Added entries:

```
172.31.14.17 master
172.31.9.88 worker1
172.31.9.239 worker2
```

Each machine's `/etc/hosts` should contain the SAME 3 lines

```
172.31.14.17 master
172.31.9.88 worker1
172.31.9.239 worker2
```

A terminal window screenshot from a machine named 'ubuntu@master'. The user has executed two commands: 'sudo vi /etc/hosts' and 'sudo cat /etc/hosts'. The output of the cat command shows the contents of the /etc/hosts file, which includes localhost entries and the three host entries for the cluster: 172.31.14.17 master, 172.31.9.88 worker1, and 172.31.9.239 worker2. The prompt is currently at the end of the last line.

```
[ubuntu@master:~$ sudo vi /etc/hosts
[ubuntu@master:~$ sudo cat /etc/hosts
127.0.0.1 localhost
::1 ip6-localhost ip6-loopback

172.31.14.17 master
172.31.9.88 worker1
172.31.9.239 worker2

ubuntu@master:~$
```

In other nodes (worker1,2) also

After adding — verify

Run on each machine:

```
ping master
ping worker1
ping worker2
```

```
[ubuntu@master:~]$ ping master
PING master (172.31.14.17) 56(84) bytes of data.
 64 bytes from master (172.31.14.17): icmp_seq=1 ttl=64 time=0.014 ms
 64 bytes from master (172.31.14.17): icmp_seq=2 ttl=64 time=0.024 ms
 64 bytes from master (172.31.14.17): icmp_seq=3 ttl=64 time=0.023 ms
 64 bytes from master (172.31.14.17): icmp_seq=4 ttl=64 time=0.030 ms
```

Step 5: Set up the IPV4 bridge on all nodes

Configure networking

Run on ALL nodes:

Load kernel modules

```
bash

sudo modprobe overlay
sudo modprobe br_netfilter
```

Make permanent:

```
bash

cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

Enable IP forwarding

```
bash

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
```

Apply settings:

```
sudo sysctl --system
```

```
ubuntu@master:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
ubuntu@master:~$ sudo modprobe overlay
ubuntu@master:~$ sudo modprobe br_netfilter
ubuntu@master:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
ubuntu@master:~$ sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
```

Similarly in other nodes also add same steps

STEP 6 — Install Container Runtime (containerd)

(Kubernetes no longer uses docker by default)

Run on all 3:

```
sudo apt update
sudo apt install -y containerd
Generate config:
```

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
Important fix (very important):
```

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup =
true/' /etc/containerd/config.toml
```

Restart:

```
sudo systemctl restart containerd
sudo systemctl enable containerd
```

```
ubuntu@master:~$ sudo apt update
sudo apt install -y containerd
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1743 kB]
Get:14 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [325 kB]
```

```
ubuntu@master:~$ sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
disabled_plugins = []
imports = []
oom_score = 0
plugin_dir = ""
required_plugins = []
root = "/var/lib/containerd"
state = "/run/containerd"
temp = ""
version = 2
```

```
ubuntu@master:~$ sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
ubuntu@master:~$ sudo systemctl restart containerd
sudo systemctl enable containerd
ubuntu@master:~$
```

—> similar in all node

STEP 7 — Install Kubernetes Packages

Run on **ALL THREE machines** (master + worker1 + worker2)

Add Kubernetes repository

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates
curl gpg
```

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/
Release.key | \
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-
keyring.gpg
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-
keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /'
| \
sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Install kubeadm, kubelet, kubectl

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

```

ubuntu@master:~$ sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.6).
curl set to manually installed.
The following additional packages will be installed:
  dirmngr gnupg gnupg-l10n gnupg-utils gpg-agent gpg-wks-client gpgconf gpgsm gpgv keyboxd
Suggested packages:
  pinentry-gnome3 tor parcimonie xloadimage gpg-wks-server scd daemon
The following NEW packages will be installed:
  apt-transport-https
The following packages will be upgraded:
  dirmngr gnupg gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client gpgconf gpgsm gpgv keyboxd
11 upgraded, 1 newly installed, 0 to remove and 73 not upgraded.

```

```

ubuntu@master:~$ curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | \
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
ubuntu@master:~$ echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ ' | \
sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /
ubuntu@master:~$ sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease

```

Verify:

```

kubectll set on hold.
ubuntu@master:~$ kubeadm version
kubectl version --client
kubeadm version: &version.Info{Major:"1", Minor:"30", GitVersion:"v1.30.14", G
Client Version: v1.30.14
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
ubuntu@master:~$

```

kubeadm version

```
kubectl version --client
```

—> similarly in other nodes (worker1,2)

STEP 8 — Initialize MASTER NODE (Most Important Step)

Run **ONLY** on MASTER machine

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
```

—> This will take 2–3 minutes

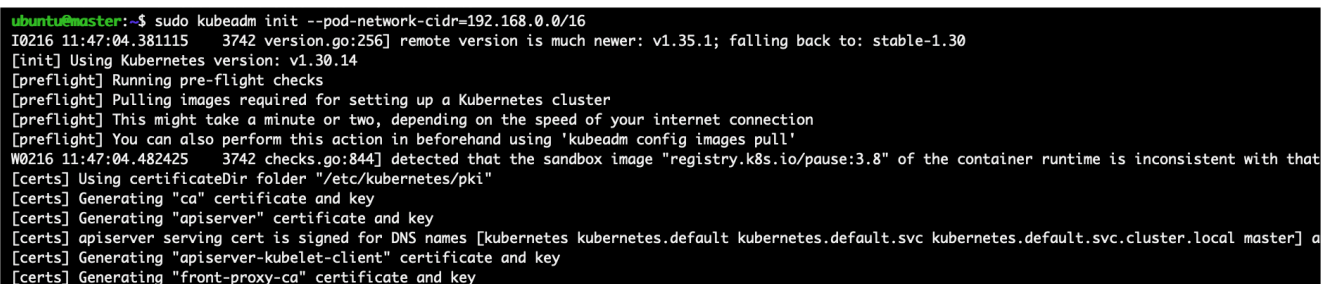
At the end it prints a LONG command called **kubeadm join**

Example:

```
kubeadm join 172.31.xx.xx:6443 --token abcdef.123456789 \
--discovery-token-ca-cert-hash sha256:xxxxxxxxx
```

—> COPY THAT COMMAND — DO NOT CLOSE TERMINAL

You will use it on workers.

A terminal window showing the execution of 'sudo kubeadm init' on a master node. The output includes version checks, preflight checks, and the generation of certificates and keys. The terminal text is as follows:

```
ubuntu@master:~$ sudo kubeadm init --pod-network-cidr=192.168.0.0/16
I0216 11:47:04.381115    3742 version.go:256] remote version is much newer: v1.35.1; falling back to: stable-1.30
[init] Using Kubernetes version: v1.30.14
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'
W0216 11:47:04.482425    3742 checks.go:844] detected that the sandbox image "registry.k8s.io/pause:3.8" of the container runtime is inconsistent with that
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [kubernetes.kubernetes.default.kubernetes.default.svc.kubernetes.default.svc.cluster.local master] a
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
```


Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 172.31.14.17:6443 --token 6tbrxd.0ka2udv4xnkpeat5 \
--discovery-token-ca-cert-hash sha256:0942aef99ead80ebd05b57f0ff5c2727857b08e81df6d565aa9186b128956cb1
ubuntu@master:~$
```

After init success (still on MASTER)

Enable kubectl on master :

Run:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $USER:$USER $HOME/.kube/config
```

```
ubuntu@master:~$ mkdir -p $HOME/.kube
ubuntu@master:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
ubuntu@master:~$ sudo chown $USER:$USER $HOME/.kube/config
ubuntu@master:~$ kubectl get nodes
```

Check:

```
kubectl get nodes
```

Output will be:

```
ubuntu@master:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE     VERSION
master      NotReady control-plane 4m53s   v1.30.14
```

```
master      NotReady
```

✓ Normal — network not installed yet

install Pod Network (Calico):

Run on MASTER:

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

Wait ~1 minute

Then check:

```
kubectl get nodes
```

Now master should become:

```
master    Ready
```

```
master    NotReady    control-plane    m3ss    v1.30.14
ubuntu@master:~$ kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
poddisruptionbudget.policy/calico-kube-controllers created
serviceaccount/calico-kube-controllers created
serviceaccount/calico-node created
configmap/calico-config created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org created
```

```
[ubuntu@master:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE    VERSION
master      Ready     control-plane    5m36s    v1.30.14
ubuntu@master:~$
```

STEP 9 — Join Worker Nodes to Cluster

Go to **worker1** machine and run:

```
sudo kubeadm join 172.31.14.17:6443 --token
6tbrxd.0ka2udv4xnkpeat5 \
--discovery-token-ca-cert-hash
sha256:0942aef99ead80ebd05b57f0ff5c2727857b08e81df6d565aa9186
b128956cb1
```

After success you will see:

This node has joined the cluster

```
ubuntu@worker1:~$ sudo kubeadm join 172.31.14.17:6443 --token 6tbrxd.0ka2udv4xnkpeat5 \
--discovery-token-ca-cert-hash sha256:0942aef99ead80ebd05b57f0ff5c2727857b08e81df6d565aa9186b128956cb1
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.620215ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

```
ubuntu@worker1:~$
i-0b424c636f70f9196 (k8s-worker1)
PublicIPs: 3.12.152.185 PrivateIPs: 172.31.9.88
```

Do the same command on **worker2** also.

```
ubuntu@worker2:~$ sudo kubeadm join 172.31.14.17:6443 --token 6tbrxd.0ka2udv4xnkpeat5 \
--discovery-token-ca-cert-hash sha256:0942aef99ead80ebd05b57f0ff5c2727857b08e81df6d565aa9186b128956cb1
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.9128ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

```
ubuntu@worker2:~$
i-0c8a37205a734d495 (k8s-worker2)
PublicIPs: 18.119.136.35 PrivateIPs: 172.31.9.239
```

Step 10:

Then go back to MASTER and verify

```
kubectl get nodes
```

You should finally see:

```
master    Ready
worker1   Ready
worker2   Ready
```

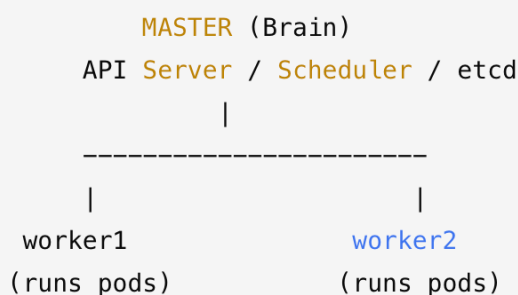
```
master    NotReady    control-plane    5m21s    v1.30.14
[ubuntu@master:~$ kubectl get nodes
NAME      STATUS    ROLES    AGE      VERSION
master    Ready     control-plane    5m36s    v1.30.14
[ubuntu@master:~$ kubectl get nodes
NAME      STATUS    ROLES    AGE      VERSION
master    Ready     control-plane    13m      v1.30.14
worker1   Ready     <none>     2m52s    v1.30.14
worker2   Ready     <none>     89s      v1.30.14
ubuntu@master:~$
```

That means:

- Master created
- Workers joined
- Networking working
- Cluster healthy

Architecture now:

arduino



What you achieved

You built a **real multi-node Kubernetes cluster on AWS** using kubeadm.

Now Task-2 Record Result

A Kubernetes cluster consisting of one master node and two worker nodes was successfully configured using kubeadm. All nodes were in Ready state and communication between nodes was verified using kubectl get nodes.

Task :3 Run one nginx pod.

STEP 1 — Create YAML file

On MASTER:

vi nginx-pod.yaml
-> add the below code

```
[ubuntu@master:~$ vi nginx-pod.yaml
[ubuntu@master:~$ cat nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80

ubuntu@master:~$
```

STEP 2 — Create Pod using YAML

```
kubectl apply -f nginx-pod.yaml
```

Verify:

```
kubectl get pods
```

```
ubuntu@master:~$ kubectl apply -f nginx-pod.yaml
pod/nginx created
ubuntu@master:~$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           10s
ubuntu@master:~$
```

STEP 3 — Add Label to Pod

Run:

```
kubectl label pod nginx app=nginx
```

Verify:

```
kubectl get pods --show-labels
```

```
nginx     1/1     Running   0           10s
ubuntu@master:~$ kubectl label pod nginx app=nginx
pod/nginx labeled
ubuntu@master:~$ kubectl get pods --show-labels
NAME      READY   STATUS    RESTARTS   AGE   LABELS
nginx     1/1     Running   0           110s   app=nginx
ubuntu@master:~$
```

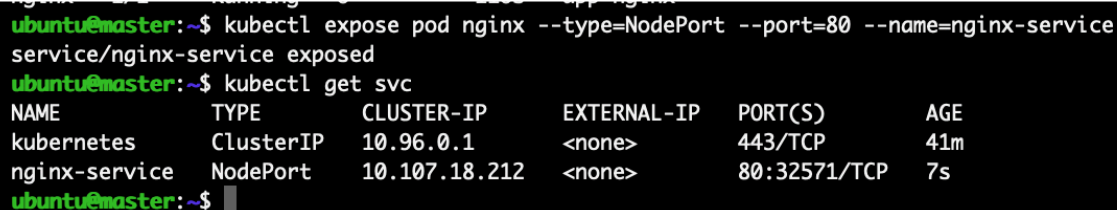
STEP 4 — Expose Pod (Create Service)

Run:

```
kubectl expose pod nginx --type=NodePort --port=80 --name=nginx-service
```

Check service:

```
kubectl get svc
```



```
ubuntu@master:~$ kubectl expose pod nginx --type=NodePort --port=80 --name=nginx-service
service/nginx-service exposed
ubuntu@master:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	41m
nginx-service	NodePort	10.107.18.212	<none>	80:32571/TCP	7s

Your service:

```
nginx-service      NodePort      10.107.18.212      80:32571/TCP
```

—> NodePort = 32571

Final Step — Open in Browser

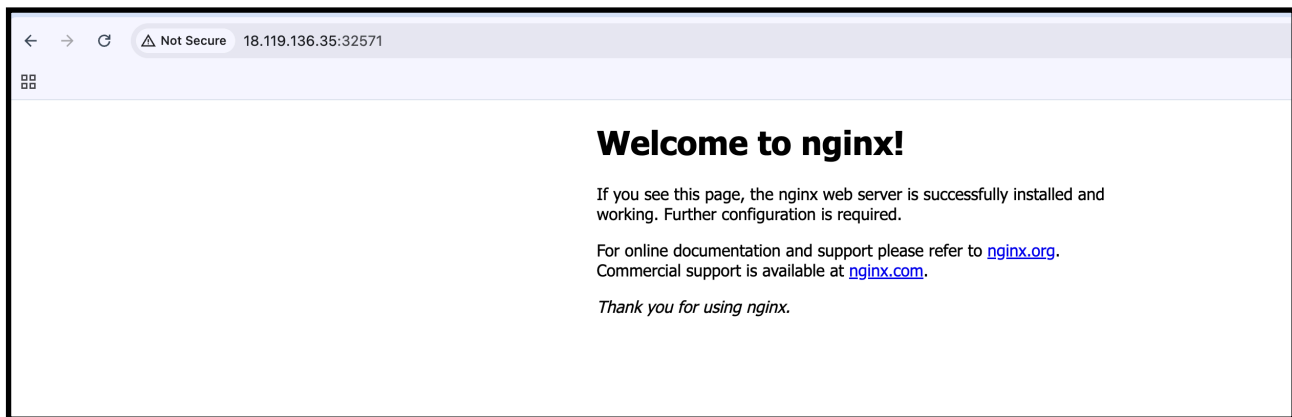
Use **ANY worker public IP** (not private 172...)

Example:

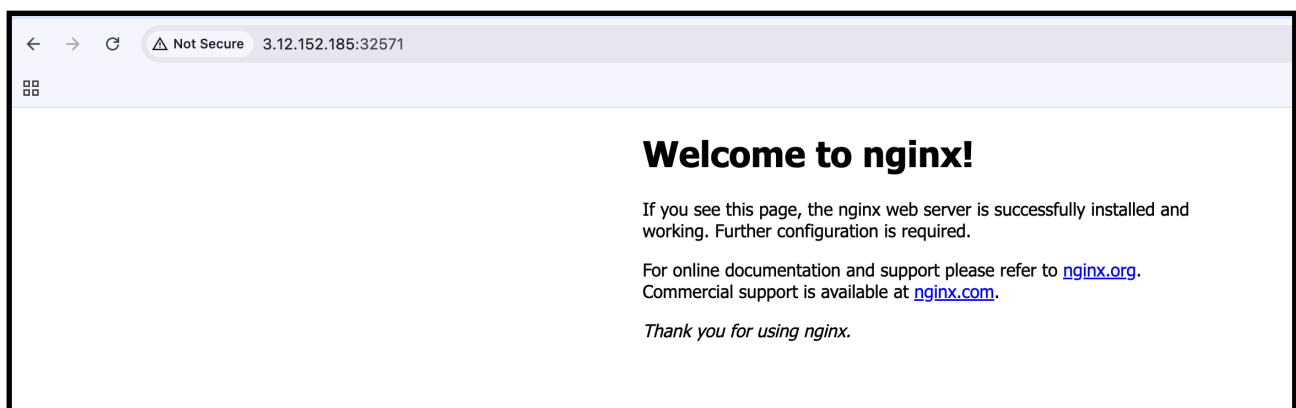
```
http://<worker-public-ip>:32571
```

```
http://18.119.136.35:32571/ (worker 2ip)
```

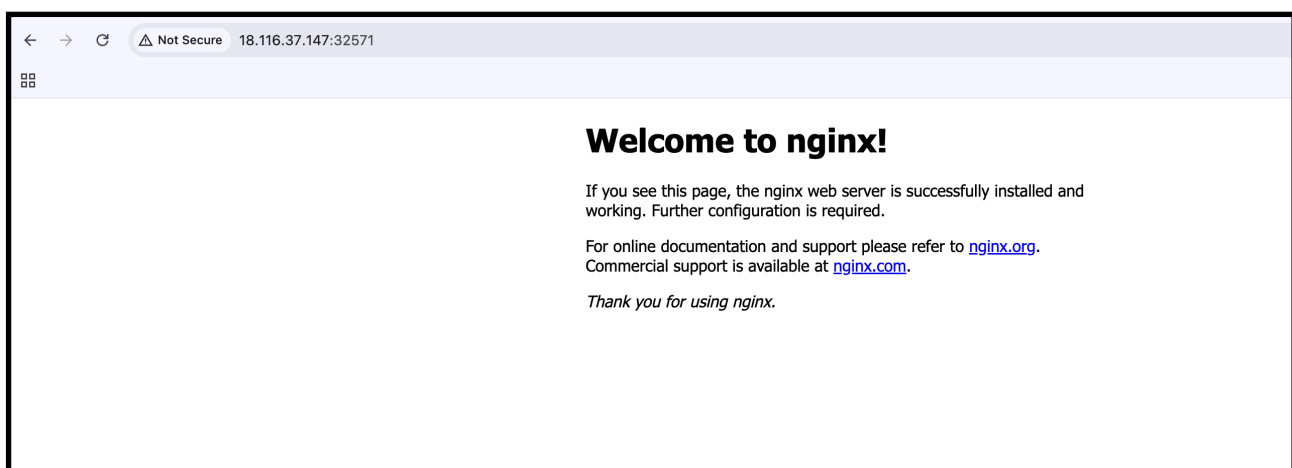
—> worker2



—> worker1



—> master



Task 4:

Mugup Master and slave components on k8s.

Kubernetes Master Node (Control Plane)

Role: Controls and manages the whole cluster

Components

1) etcd

- Key-value database of Kubernetes
- Stores cluster state (pods, nodes, configs, secrets)
- Called the *brain* because cluster depends on it

2) kube-apiserver

- Entry point of Kubernetes
- All commands go through API server

3) kube-scheduler

- Selects which worker node will run a pod
- Assigns pod to best available node

4) controller-manager

- Monitors cluster and maintains desired state
- Restarts failed pods
- Recreates deleted containers

Master in one line

Master node manages the cluster and makes decisions using etcd data.

Kubernetes Worker Node (Slave Node)

Role: Runs the actual applications (pods/containers)

Components

1) kubelet

- Communicates with master
- Creates and manages pods on the node

2) container runtime (containerd)

- Runs containers inside pods

3) kube-proxy

- Handles networking and load balancing
- Allows pods and services to communicate

Worker in one line

Worker node executes application workloads instructed by master.

Very Short Memory Trick

Master → **A S C E**

- A → API Server
- S → Scheduler
- C → Controller Manager
- E → etcd (brain)

Worker → **K P C**

- K → Kubelet
- P → Proxy
- C → Container runtime

